

EXPERIMENT 23

By Lakshya Yadav

Aim

To develop a menu-driven C program using a switch statement and nested loops, without arrays, to print the selected square/rectangular pattern in a separate parameterless function that returns no value.

Operations

1. Solid square pattern - input n and print n rows and n columns of *.
2. Solid rectangular pattern - input rows m and columns n and print an m x n rectangle of *.
3. Row-number pattern - print an m x n pattern where every element in a row equals its row number.
4. Column-number pattern - print an m x n pattern where every element in a column equals its column number.
5. Exit.

Algorithm

1. Start.
2. Display the menu repeatedly using a do-while loop.
3. Read the user's choice.
4. Use switch-case to call the corresponding parameterless function.
5. In each function, read the required positive dimensions; if invalid, display an error and request them again.
6. Use nested loops to print the requested pattern.
7. For an invalid menu choice, display an appropriate error message.
8. Continue until Exit is selected.
9. Stop.

Test-Case Table

TC	Input	Expected Result
1	Choice 1, n=4	4 x 4 solid square of *
2	Choice 2, m=3, n=5	3 x 5 solid rectangle of *
3	Choice 3, m=4, n=4	Rows: 1111, 2222, 3333, 4444
4	Choice 4, m=4, n=4	Each row: 1 2 3 4
5	Choice 1, n=1	Single * (boundary valid input)
6	Choice 2, m=0 then 2, n=-1 then 3	Reject invalid dimensions; accept 2 and 3
7	Choice 9	Display invalid menu choice message
8	Choice 5	Exit program

C Program

```
#include <stdio.h>

void solidSquare(void);
void solidRectangle(void);
void rowNumberPattern(void);
void columnNumberPattern(void);

int main(void)
{
    int choice;
    do {
        printf("\n--- PATTERN MENU ---\n");
        printf("1. Solid square pattern\n");
        printf("2. Solid rectangular pattern\n");
```

```

        printf("3. Row-number pattern\n");
        printf("4. Column-number pattern\n");
        printf("5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1: solidSquare(); break;
            case 2: solidRectangle(); break;
            case 3: rowNumberPattern(); break;
            case 4: columnNumberPattern(); break;
            case 5: printf("Exiting program.\n"); break;
            default: printf("Invalid menu choice. Try again.\n");
        }
    } while (choice != 5);
    return 0;
}

void solidSquare(void)
{
    int n, i, j;
    do {
        printf("Enter number of rows/columns n: ");
        scanf("%d", &n);
        if (n <= 0) printf("Error: n must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= n; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void solidRectangle(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m: ");
        scanf("%d", &m);
        if (m <= 0) printf("Error: m must be positive.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n: ");
        scanf("%d", &n);
        if (n <= 0) printf("Error: n must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("* ");
        printf("\n");
    }
}

void rowNumberPattern(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m: ");
        scanf("%d", &m);
        if (m <= 0) printf("Error: m must be positive.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n: ");
        scanf("%d", &n);
        if (n <= 0) printf("Error: n must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", i);
        printf("\n");
    }
}

```

```

void columnNumberPattern(void)
{
    int m, n, i, j;
    do {
        printf("Enter rows m: ");
        scanf("%d", &m);
        if (m <= 0) printf("Error: m must be positive.\n");
    } while (m <= 0);
    do {
        printf("Enter columns n: ");
        scanf("%d", &n);
        if (n <= 0) printf("Error: n must be positive.\n");
    } while (n <= 0);

    for (i = 1; i <= m; i++) {
        for (j = 1; j <= n; j++)
            printf("%d ", j);
        printf("\n");
    }
}

```

Compilation and Execution

```

gcc experiment23.c -o experiment23
./experiment23

```

Sample Verification

```

Choice 1, n = 4:
* * * *
* * * *
* * * *
* * * *

```

```

Choice 3, m = 4, n = 4:
1 1 1 1
2 2 2 2
3 3 3 3
4 4 4 4

```

```

Choice 4, m = 4, n = 4:
1 2 3 4
1 2 3 4
1 2 3 4
1 2 3 4

```

Result

The menu-driven C program was implemented using switch-case, nested loops and separate parameterless functions. It correctly prints all four specified patterns and handles invalid menu choices and non-positive dimensions.